CycleRL: Sim-to-Real Deep Reinforcement Learning for Robust Autonomous Bicycle Control

Gelu Liu, Teng Wang, Zhijie Wu, Junliang Wu, Songyuan Li, Xiangwei Zhu

arXiv:2603.15013v2 [cs.RO] 3 May 2026

***Abstract*—Autonomous bicycles offer a promising agile solution for urban mobility and last-mile logistics. However, conventional control strategies often struggle with underactuated nonlinear dynamics, suffering from sensitivity to model mismatches and limited adaptability to real-world uncertainties. To address this, we develop CycleRL, a comprehensive sim-to-real framework for robust autonomous bicycle control. Our approach establishes a direct perception-to-action mapping within the high-fidelity NVIDIA Isaac Sim environment, leveraging Proximal Policy Optimization (PPO) to optimize the control policy. The framework features a composite reward function tailored for concurrent balance maintenance, velocity tracking, and steering control. Crucially, systematic domain randomization is employed to reduce the reliance on precise system modeling, bridge the simulation-to-reality gap and facilitate direct transfer. In simulation, CycleRL achieves promising performance, including a 99.90% balance success rate, a heading tracking error of 1.15°, and a velocity tracking error of 0.18 m/s. These quantitative results, coupled with successful hardware deployment, validate DRL as an effective paradigm for autonomous bicycle control, offering superior adaptability over traditional methods. Video demonstrations are available at https://anony6f05.github.io/CycleRL/.**

***Index Terms*—Deep reinforcement learning, autonomous bicycle control, sim-to-real transfer, domain randomization, underactuated systems, lateral control.**

I. Introduction

The autonomous control of two-wheeled vehicles is a classic challenge in robotics, involving stabilizing inherently unstable dynamics while executing steering maneuvers [1]. Such capability is critical for emerging applications such as autonomous cycling, micro-logistics, and intelligent mobility solutions, which demand robust stability and precise lateral control in complex urban environments [2]. Consequently, autonomous bicycle control has become a key testbed for advancing control methodologies and embodied intelligence.

Two primary paradigms exist for self-balancing systems: reaction wheels and steering control. While reaction-wheel-based approaches offer static balancing capabilities [3], [4], they are often limited by high energy consumption, increased mechanical complexity, and payload constraints. In contrast, steering control presents a more pragmatic and efficient alternative that aligns with natural bicycle dynamics [5], [6]. Its simplified architecture, lower energy requirements, and superior load-carrying capacity make it the preferred choice for practical, real-world applications.



Fig. 1: Real-world deployment of the proposed autonomous bicycle control framework across different conditions.

Historically, steering control has been implemented via classical methods, ranging from PID controllers [7], [8], LQR [3], MPC [9], LPV [1], to fuzzy logic [10]. However, these approaches face significant drawbacks. PID controllers demand extensive manual tuning, while model-based methods, such as MPC, are computationally intensive and highly sensitive to inaccuracies in system parameters. This reliance on precise models often results in controllers that are brittle to real-world variations, compromising their reliability.

These limitations have motivated the shift toward Deep Reinforcement Learning (DRL), a powerful paradigm for solving complex sequential decision-making problems with high-dimensional inputs and nonlinear dynamics [11]. By leveraging high-fidelity simulators, such as Isaac Gym [12] and MuJoCo [13], DRL enables agents to learn optimal control strategies through autonomous interaction, thereby reducing the reliance on highly accurate analytical models. This paradigm establishes a direct mapping from sensory observations to actuator commands, which can reduce system latency and optimize the control pipeline, offering superior robustness and adaptability for dynamic environments [14].

In this work, we present CycleRL, a model-free deep reinforcement learning (DRL) framework for autonomous bicycle control that reduces the reliance on accurate analytical dynamics models. Trained within Isaac Sim, our approach designed a

This work was supported by the National Natural Science Foundation of China under Grant No. T2350005 and No. T2541041, the Fundamental Research Funds for the Central Universities, Sun Yat-sen University under Grant No. 23xkjc008. This work has been submitted to the IEEE for possible publication. Copyright may be transferred without notice, after which this version may no longer be accessible. (Corresponding author: Xiangwei Zhu.)

Gelu Liu, Teng Wang, Zhijie Wu, Junliang Wu, Songyuan Li, Xiangwei Zhu are with the School of Electronics and Communication Engineering and Shenzhen Key Laboratory of Navigation and Communication Integration, Sun Yat-sen University, Shenzhen 518107, China (e-mail: liuglu@mail2.sysu.edu.cn; wangt556@mail.sysu.edu.cn; wuzhj39@mail2.sysu.edu.cn; wujliang9@mail2.sysu.edu.cn; lisy287@mail.sysu.edu.cn; zhuxw666@mail.sysu.edu.cn).

0000–0000/00$00.00 © 2026 IEEE

tailored composite reward function to simultaneously optimize balance, velocity, and steering. To bridge the reality gap, we implement a comprehensive domain randomization strategy that facilitates reliable sim-to-real transfer. Extensive evaluations demonstrate that CycleRL achieves a 99.90% simulated balance success rate—outperforming classical baselines—and exhibits exceptional real-world robustness on a custom physical platform against diverse conditions, as shown in Fig. 1.

The remainder of this paper is structured as follows: Section II reviews related work in autonomous control and DRL. Section III details our methodological framework. Section IV presents our experimental evaluations, and Section V concludes the paper. Our primary contributions are:

- A comprehensive system integration of a model-free Deep Reinforcement Learning (DRL) framework for autonomous bicycle control, which alleviates the dependence on accurate analytical dynamics models required by conventional methods.
- A carefully engineered composite reward function coupled with a multi-layered domain randomization strategy, enabling favorable simulated control performance and reliable direct sim-to-real transferability.
- A successful sim-to-real transfer on a custom-built bicycle platform, with rigorous real-world validation demonstrating robust and promising balance performance under various physical conditions.

II. Related Work

A. Traditional Control Methods for Two-Wheeled Vehicles

Traditional model-based methodologies, including Proportional-Integral-Derivative (PID) [7], Linear-Quadratic Regulator (LQR) [3], and Model Predictive Control (MPC) [9], have been extensively explored for two-wheeled vehicle control. While these methods improve responsiveness and constraint handling, they fundamentally struggle with parameter sensitivity and lack robustness against unmodeled dynamics.

Alternatively, knowledge-based approaches, such as fuzzy logic, manage system nonlinearities without requiring a precise mathematical model. Fuzzy controllers have been successfully designed to achieve robust balance and steering control under different conditions [10], [15]. However, their performance is highly dependent on extensive expert knowledge and manual tuning, which can result in suboptimal control strategies.

Despite their foundational contributions, these traditional methods generally share common limitations: high sensitivity to model inaccuracies, significant computational demand, and limited adaptability across diverse conditions. These challenges motivate the exploration of learning-based approaches that can derive robust control policies directly from environmental interactions.

B. Learning-Based Applications in Bicycle Control

Learning-based approaches, notably imitation learning (IL) and deep reinforcement learning (DRL), have emerged as transformative paradigms in embodied control [2]. Advances in simulators, e.g., NVIDIA Isaac Gym [12] and MuJoCo [13], have enabled massive parallel experience collection and efficient policy training, lowering the barrier for deploying neural controllers in nonlinear and underactuated systems.

IL has shown promise in bicycle motion control, primarily by leveraging expert demonstrations to achieve rapid convergence in balancing and trajectory tracking. For instance, Weissmann et al. used conditional IL for vision-based bicycle trajectory prediction, achieving favorable short-term accuracy [16]. However, IL policies tend to generalize poorly beyond the training distribution and often require high-quality demonstrations, which are hard to acquire in rare scenarios, making IL less suitable for bicycle control in the real world.

In contrast, DRL enables learning control policies through trial-and-error interactions without relying on expert data, making it highly suitable for dynamically coupled vehicle control [17]. For instance, DRL has been employed to integrate multiple control objectives such as path tracking and stability in autonomous scooter driving scenarios [18], and to achieve robust navigation for vehicles traversing rough and varied terrains [19]. Focusing on the particularly challenging domain of two-wheeled vehicles, Guo et al. developed an RL algorithm based on the nonaffine nonlinear dynamics of a bicycle, successfully demonstrating self-balancing control with preliminary physical validation [20].

Despite significant progress in simulation, achieving robust sim-to-real transfer for bicycles controlled solely through steering (i.e., without reaction wheels) remains a major engineering challenge. This difficulty stems from: (1) the system's inherent instability, particularly at low speeds, and (2) its extreme sensitivity to minute physical discrepancies between the simulated and real-world models. This represents a substantial gap in advancing autonomous mobility solutions.

C. Simulation-to-Reality Transfer Technologies

A central challenge in deploying learning-based controllers is bridging the "simulation-to-reality" (sim-to-real) gap, where policies trained in simulation exhibit a significant performance degradation on physical systems [21]. This performance drop is attributable to the discrepancies between the simulated model and real-world dynamics, such as parametric inaccuracies, unmodeled stochastic dynamics, actuator discrepancies, sensor noise, and external disturbances.

Key strategies have been developed to bridge this gap. Domain randomization enhances policy generalization by training across a wide distribution of simulation parameters, a technique proven effective for transferring vision-based policies to real robots [22]. Other prominent methods include training ensembles of dynamics models to capture uncertainty, using adversarial training frameworks such as Robust Adversarial Reinforcement Learning (RARL) to prepare for worst-case scenarios [23], and employing domain adaptation to align simulation and real-world distributions [24].

However, the application of these sim-to-real techniques to the complex dynamics of bicycle control remains largely unexplored. This presents a significant opportunity to advance the real-world deployment of autonomous two-wheeled vehicles, given their unique stability challenges and high sensitivity to environmental factors.

III. METHODOLOGY

A. Nonlinear Dynamics and Uncertainty Analysis

1) Two-Wheeled Vehicle Dynamics: Two-wheeled vehicles represent a canonical class of underactuated nonholonomic systems characterized by intricate coupling between rotational and translational motion that poses a significant challenge to conventional linear control synthesis.

Consider the configuration space parameterized by generalized coordinates $q = [\varphi, \delta, x, y, \psi]^T$, where φ represents the roll angle, δ denotes the steering angle, (x, y) defines the planar position, and ψ is the yaw angle, as shown in Fig. 2. The complete nonlinear dynamics can be expressed as:

$$M_\theta(q)\ddot{q} + C_\theta(q, \dot{q})\dot{q} + G_\theta(q) = B(q)u + \tau_{\text{ext}}(t) \tag{1}$$

where $M_\theta(q) \in \mathbb{R}^{5\times5}$ is the configuration-dependent inertia matrix, $C_\theta(q, \dot{q})$ captures velocity-dependent Coriolis and gyroscopic effects, $G_\theta(q)$ represents gravitational and centrifugal forces, $B(q)$ is the input distribution matrix, and $\tau_{\text{ext}}(t)$ denotes external disturbances [25].

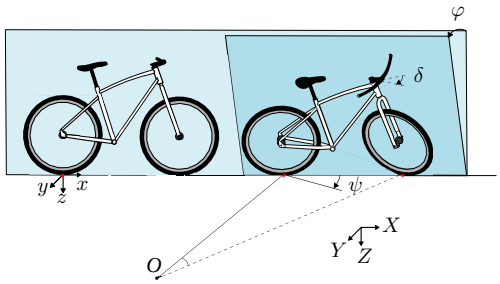


Fig. 2: Sketch of the two-wheeled vehicle [26].

2) Stochastic Dynamics Formulation: To capture real-world uncertainties, we model the bicycle's system evolution via a stochastic differential equation (SDE):

$$dq = f_\theta(q, u)dt + g_\theta(q)dW_t + h_\theta(q)d\xi_t \tag{2}$$

where θ denotes uncertain physical parameters (e.g., mass distribution, friction). Here, $f_\theta(q, u)$ governs the nominal nonlinear dynamics, while environmental stochasticity is captured by two terms: $g_\theta(q)dW_t$ models continuous disturbances (e.g., terrain vibrations) via a standard Wiener process W_t, and $h_\theta(q)d\xi_t$ models discrete impulsive events (e.g., sudden road steps) via a Poisson jump process ξ_t [27].

Analytically solving the optimal control problem for Eq. 2 is mathematically intractable. While traditional approaches like nonlinear MPC can approximate solutions, they typically require stringent simplifying assumptions, exact explicit system identification, or computationally prohibitive online optimization. To bypass these modeling and computational bottlenecks, we adopt a model-free DRL framework, as detailed below.

B. Reinforcement Learning Framework

1) Markov Decision Process Formulation: We formulate the control problem as a Markov Decision Process (MDP), defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$. Here, $\mathcal{S} \in \mathbb{R}^n$ and $\mathcal{A} \in \mathbb{R}^m$ denote the continuous state and action spaces, respectively. $\mathcal{P}$ governs the transition dynamics, $\mathcal{R}$ defines the control objectives, and γ is the discount factor. Although the physical system is partially observable, we utilize the instantaneous observation vector o_t (comprising IMU and encoder data) directly as the state s_t. This approximation is justified by the high control frequency, which mitigates the need for temporal history integration. Consequently, we learn a reactive stochastic policy $\pi_\theta(a_t|s_t)$ parameterized by a standard Multi-Layer Perceptron (MLP), relying on domain randomization to ensure robustness against sensor noise.

2) Composite Reward Function Design: The controller's learning process is guided by a composite reward function designed to balance performance goals with control efficiency (illustrated in Fig. 3). By aggregating positive incentives (for stability and tracking) and negative penalties (for action smoothness) into a single scalar signal, we transform the multi-objective requirements into a tractable optimization problem.

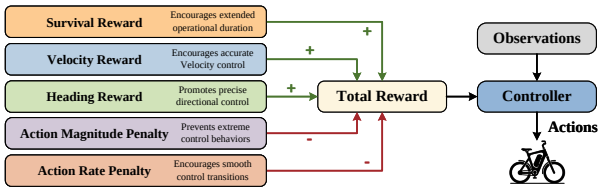


Fig. 3: Illustration of the reward function design for balancing and steering control. The total reward aggregates performance incentives (green arrows) and control effort penalties (red arrows) into a unified scalar to guide the policy learning.

The total reward R_t at each timestep is computed as the weighted sum of five key components:

$$\begin{aligned} R_t = \lambda_{\text{surv}} \cdot r_{\text{surv}} + \lambda_{\text{vel}} \cdot r_{\text{vel}} + \lambda_{\text{head}} \cdot r_{\text{head}} + \\ \lambda_{\text{act}} \cdot r_{\text{act}} + \lambda_{\text{rate}} \cdot r_{\text{rate}} \end{aligned} \tag{3}$$

where the weights are set to $\lambda_{\text{surv}} = 1.0$, $\lambda_{\text{vel}} = 3.0$, $\lambda_{\text{head}} = 5.0$, $\lambda_{\text{act}} = 1.0$, and $\lambda_{\text{rate}} = 2.0$. These components are organized into two groups: performance-oriented rewards and control effort penalties.

a) *Performance-Oriented Rewards*: These components provide positive reinforcement for successfully executing the primary balancing and tracking control tasks.

Survival Reward (r_{surv}): To encourage long-term stability and faster convergence, a constant positive reward is given for each timestep the bicycle remains operational. This incentivizes the agent to avoid falling:

$$r_{\text{surv}} = 1 \tag{4}$$

Velocity Tracking Reward (r_{vel}): This reward encourages the agent to accurately follow the target forward velocity (v_{cmd}). The reward is exponentially proportional to the negative error between the actual and commanded velocities:

$$r_{\text{vel}} = \exp(-\alpha|v_{\text{actual}} - v_{\text{cmd}}|) \tag{5}$$

where the error sensitivity $\alpha = 0.25$.

Heading Tracking Reward (r_{head}): Similarly, this component rewards the agent for precise directional control by minimizing the heading error between the actual (ψ_{actual}) and desired (ψ_{desired}) heading, where $\beta = 0.1$:

$$r_{\text{head}} = \exp(-\beta|\psi_{\text{actual}} - \psi_{\text{desired}}|) \tag{6}$$

b) *Control Effort Penalties*: These components are negative rewards designed to regularize the control policy, leading to smoother and more efficient actions.

Action Magnitude Penalty (r_{act}): A penalty proportional to the L2 norm of the action (a_t) is applied to discourage aggressive policy outputs, which promotes energy efficiency:

$$r_{\text{act}} = -\|a_t\|_2 \tag{7}$$

Action Rate Penalty (r_{rate}): To ensure smooth control outputs, a penalty is imposed on the L2 norm of the change in action between consecutive timesteps. This discourages high-frequency oscillations (jerkiness):

$$r_{\text{rate}} = -\|a_t - a_{t-1}\|_2 \tag{8}$$

While the DRL approach avoids deriving explicit analytical controllers, the pipeline still involves important engineering decisions, particularly in reward shaping. The hyperparameters were systematically selected based on a hierarchical control logic and sensitivity analysis (detailed in Section IV-E). Tracking rewards ($\lambda_{\text{vel}}, \lambda_{\text{head}}$) are assigned higher weights than the survival baseline to prevent the agent from converging to a trivial "stand-still" policy. Additionally, the steering sensitivity is set with a tighter tolerance ($\beta < \alpha$) due to the critical coupling between lateral deviations and roll instability, while penalty weights are calibrated to constrain control frequencies within the physical actuator bandwidth.

c) *Episode Termination Criteria*: Two distinct termination conditions are implemented to balance training efficiency with safety considerations. Timeout termination limits episodes to 64 seconds (3,200 steps at 50 Hz), while safety-based termination immediately ends episodes when $|\varphi_t| > 45°$, providing clear failure signals while preventing continuation in physically unrealistic states.

3) Proximal Policy Optimization with Composite Reward: We employ the Proximal Policy Optimization (PPO) [28] algorithm to train the control policy. To handle multiple control objectives, the weighted composite reward R_t is linearly scalarized into a unified signal for computing the Generalized Advantage Estimation (GAE). The policy π_θ is then optimized to maximize the expected cumulative return using the standard clipped surrogate objective:

$$\mathcal{L}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right] \tag{9}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ represents the probability ratio between the current and old policies, $\hat{A}_t$ denotes the advantage estimate derived from the composite reward R_t, and ϵ is the clipping threshold that constrains the policy ratio to the range $[1-\epsilon, 1+\epsilon]$, thereby ensuring training stability.

C. Domain Randomization Strategy

Sim-to-real discrepancies—ranging from unmodeled dynamics to unseen states—often cause catastrophic hardware failures [22]. To bridge this gap, we employ a decoupled hierarchical randomization strategy comprising *Dynamics*, *Initial State*, and *Task Randomization*, as shown in Table I. Tailored to underactuated bicycles, this multi-layered approach prevents overfitting to simulation artifacts and forces the policy to generalize across diverse physical conditions.

TABLE I: Domain Randomization Variables

Randomization Variables	Range/Distribution
Dynamics Randomization (Physical Parameters)	
System Total Mass (m_{total})	$\mathcal{U}([15.0, 45.0])$ kg
Center of Mass Height (h_{CoM})	$\mathcal{U}([0.50, 0.80])$ m
Tire Friction Coefficient (μ)	$\mathcal{U}([0.5, 1.2])$
Actuator Force Gain	$\mathcal{U}([0.9, 1.1])$
Observation Noise	$\mathcal{N}(0,\ (\rho \cdot s_{\max})^2)$, $\rho \sim \mathcal{U}([0.01, 0.20])$
Initial State Randomization	
Initial Body Velocity (v_{init})	$\mathcal{U}([1.0, 2.5])$ m/s
Initial Body Lean Angle (φ_{init})	$\mathcal{U}([-10, 10])$ deg
Initial Servo Motor Position ($\theta_{m,\text{init}}$)	$\mathcal{U}([-20, 20])$ deg
Initial Hub Motor Velocity ($\omega_{m,\text{init}}$)	$\mathcal{U}([0, 3.0])$ rad/s
Task / Command Randomization	
Target Velocity Command (v_{cmd})	$v_{\text{cmd}} \sim \mathcal{U}([1.0, 5.0])$ m/s, $t_{\text{resample}} \sim \mathcal{U}([3, 5])$ s
Target Heading Command (δ_{cmd})	$\delta_{\text{cmd}} \sim \mathcal{U}([-10, 10])$ deg, $t_{\text{resample}} \sim \mathcal{U}([3, 5])$ s

1) Dynamics Randomization: To address parametric uncertainties and modeling inaccuracies, we randomize key physical properties (see Table I). Specifically, mass, center-of-mass height and terrains are varied to reflect configuration changes, while friction and actuator gains are randomized to capture diverse terrains and hardware variations. Additionally, Gaussian noise (scaled by signal magnitude $s_{\max}$) is injected to simulate sensor imperfections, ensuring distributional robustness.

2) Initial State Randomization: We randomize the bicycle's initial kinematic states at the start of each episode. This forces the agent to learn policies that can recover from diverse unstable configurations (e.g., varying initial speeds and tilt angles) rather than relying on a static, upright start, resiliently ensuring safety during real-world engage-disengage transitions.

3) Task and Command Randomization: To develop a versatile controller, target velocity and heading commands are dynamically altered at random time intervals during training. This compels the policy to learn a generalizable state-command mapping rather than overfitting to a static equilibrium, ensuring reliable high-level directive tracking.

D. System Architecture and Implementation

1) Physical Simulation Framework: The training environment utilizes NVIDIA Isaac Sim for GPU-accelerated, parallel physics simulation. It explicitly incorporates advanced contact and actuator dynamics, variable surface friction, and realistic IMU sensor properties. This comprehensive modeling exposes the policy to complex real-world vehicle-terrain nonlinearities, as illustrated in Fig. 4. Detailed vehicle kinematics and dynamic parameters are provided in our supplementary video.

2) Hierarchical Control Architecture: To support real-time onboard deployment, we designed a hierarchical architecture consisting of input processing, policy inference, and actuator control layers. An asynchronous multi-process framework efficiently manages the high-frequency data flow, ensuring minimal end-to-end latency for the 50 Hz control loop. Further details regarding the software stack and signal flow are available in the supplementary video.

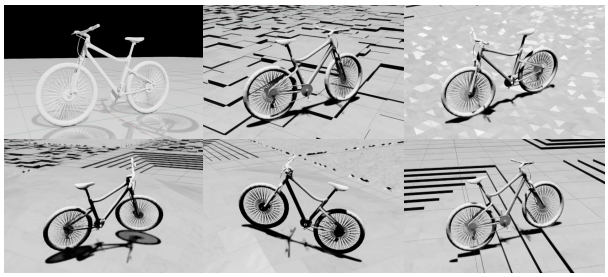


Fig. 4: Bicycle and terrain modeling in Isaac Sim.

IV. Experiments

A. Evaluation Metrics and Implementation Details

1) Metrics Definition on Performance: We establish a comprehensive evaluation framework assessing three critical dimensions for autonomous bicycle control, as stated in Table II.

TABLE II: Evaluation Metrics on Performance

Metric	Definition
Balance Metrics	
Balance Success Rate (BSR)	Percentage of episodes maintaining $\|\varphi\| < 0.5$ rad.
Balance Recovery Time (BRT)	Recovery time from disturbances exceeding 0.3 rad.
Maximum Balance Duration (MBD)	Longest continuous balancing period achieved.
Critical Angle Tolerance (CAT)	Maximum recoverable roll angle maintaining BSR $>$ 95%.
Control Metrics	
Heading Tracking Error (HTE)	RMS error between commanded and actual heading.
Velocity Tracking Error (VTE)	Mean absolute error in longitudinal velocity control.
System Response Latency (SRL)	Time to reach within 10% of the target command value.
Robustness Metrics	
Maximum Noise Tolerance (MNT)	Highest sensor noise level maintaining BSR $>$ 95%.
Minimum Sustaining Speed (MSS)	Minimum speed maintaining BSR $>$ 95%.

2) Training Configuration and Hyperparameters: Training details, including configurations and hyperparameters, are provided in supplementary video. Our training infrastructure leverages NVIDIA Isaac Sim, achieving over 700,000 simulation steps per second across 16,384 parallel environments on NVIDIA RTX 4090, ensuring a considerable iteration speed.

B. Simulation Experiments and Quantitative Analysis

1) Evaluation on Simulation Performance: We conduct comprehensive experiments across 10,000 distinct episodes to establish core performance characteristics. Evaluations are performed under ideal dynamics (i.e., no physical parameter randomization) but retain initial state and command randomization, with target velocities sampled from $\mathcal{U}([2.0, 3.0])$ m/s for our policy and fixed at 2.0 m/s for baselines.

To validate the proposed approach, we benchmark against calibrated PID and LQR baselines, which serve as canonical benchmarks for self-balancing systems and, comparable to CycleRL, require only limited system identification. PID baseline features a dual-loop structure (steer/balance) with velocity-dependent gain scaling and base gains were tuned to $K_p = 4.0$ and $K_d = 0.4$ via Ziegler-Nichols at 2.0 m/s. LQR baseline utilizes real-time Riccati equation solving on a linearized model, whose cost weights were set to $Q = \text{diag}[20.0, 6.0, 3.5]$ and $R = 1.5$, with heading tracking achieved by mapping δ_{cmd} to an equilibrium roll angle. We also implement the advanced Modified Linear Control Law (MLCL) [29] for a stronger and comparable model-based evaluation. Table III presents the quantitative results across our evaluation metrics.

TABLE III: Comparative Simulation Performance

Metric [1]	PID	LQR	MLCL	Ours
Balance Performance				
Success Rate (%) ↑	76.50	93.24	96.43	**99.90**
Recovery Time (s) ↓	1.41$\pm$0.22	1.47$\pm$0.21	1.21$\pm$0.04	**1.05**$\pm$**0.18**
Max. Duration (s) ↑	15.0	148.9	603.2	**>1,800**
Crit. Angle Tol. (°) [2] ↑	22.24	21.40	24.65	**27.79**
Control Performance				
Heading Error (°) ↓	4.04$\pm$1.51	1.82$\pm$0.64	2.87$\pm$1.66	**1.15**$\pm$**0.72**
Velocity Error (m/s) [3] ↓	†	†	†	**0.18**$\pm$**0.26**
Response Latency (s) ↓	1.67$\pm$1.12	1.27$\pm$0.61	1.46$\pm$1.10	**1.06**$\pm$**0.17**
Robustness Indicators				
Max. Noise Tol. [4] ↑	†	†	†	**0.32**
Min. Sustaining Spd (m/s) ↓	1.84	1.71	1.50	**1.05**

[1] Reported as Mean $\pm$ Standard Deviation where applicable; † means not applicable.
[2] Reported as the minimum of both sides (conservative estimate).
[3] Baselines are tested primarily for lateral balance at constant velocities.
[4] MNT represents the max tolerable noise amplitude fraction.

The results confirm our controller's superior performance compared to traditional methods. CycleRL achieves a 99.90% BSR, significantly outperforming the PID (76.50% BSR), LQR (93.24% BSR) and MLCL (96.43% BSR) baselines. Remarkably, CycleRL demonstrates faster stabilization, recovering from perturbations in 1.05 s (approximately 25.53%, 28.57% and 13.22% faster than PID, LQR and MLCL, respectively), while tolerating extreme roll angles up to 27.79° (vs. 21.40° for LQR and 24.65° for MLCL). Furthermore, unlike the baselines which struggle at lower velocities, our controller maintains robust stability down to 1.05 m/s, exhibiting precise heading tracking with minimal heading error of 1.15°.

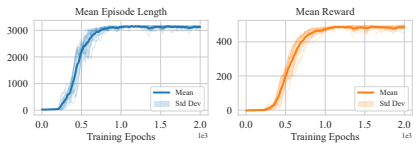
C. Convergence Analysis and Learning Dynamics

The policy's training progression under 10 random seeds is depicted in Fig. 5. Stable convergence is achieved after approximately 1,000 training epochs, equivalent to 700M simulation steps. As shown in Fig. 5a, the monotonic increase and eventual plateau of both Mean Episode Length and Mean Total Reward confirm a stable and effective learning process.

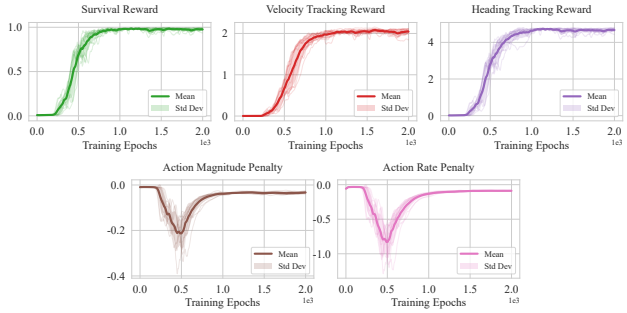
The learning dynamics, particularly the penalty curves in Fig. 5b, reveal a natural two-phase curriculum learning effect. An initial phase of aggressive control, marked by a dip in the penalty values, allows the agent to quickly master basic stability. This is followed by a refinement phase, where the penalties decrease (rise toward zero) as the agent learns to perform the task with smoother, more efficient actions.

D. Robustness Verification Experiments

To assess deployment readiness, we evaluate robustness by individually introducing velocity variations, sensor degra-



(a) Mean Episode Length & Mean Total Reward



(b) Training Rewards Curves

Fig. 5: Training curves and convergence analysis.

dation, and terrain disturbances to the baseline setup (Section IV-B1) under 10,000 trials. Table V summarizes the performance under various conditions, where the "No noise" and "Flat plane" cases replicate the ideal baseline conditions.

The evaluation confirms the system's performance and resilience. Considerable performance is achieved on ideal sensor and terrain configurations at low speeds (99.49% BSR) and at higher speeds (86.21% BSR). The system effectively handles sensor degradation, maintaining a BSR of 96.38% with severe noise and 90.84% with intermittent dropouts. On complex terrains, the system exhibits a graceful performance degradation ranging from 7.85% on slopes to 23.30% over gentle steps, compared to the ideal condition. Balance Recovery Time (BRT) and tracking errors (HTE, VTE) also remain well-maintained across diverse conditions.

Notably, a decline in BSR is observed at higher velocities. Although a bicycle exhibits enhanced passive self-stability during high-speed straight-line driving, this degradation stems from the compounding complexities of active high-speed maneuvering. Under our evaluation protocol featuring randomized velocity and heading commands, the system is potentially forced into sharp turns at high speeds. In this regime, lateral acceleration scales quadratically with velocity ($a_y \propto v^2\delta$), rendering the closed-loop system hypersensitive to steering inputs and actuator phase lag, and severely intensifying the multi-objective coupling between heading tracking and roll stability. Moreover, the fixed 50 Hz control bandwidth, coupled with the deficiencies of the PhysX-based Isaac Sim environment in simulating high-speed physical dynamics and collisions, further exacerbates the brittleness of high-speed cornering.

E. Ablation Studies

1) Ablation Study of Reward Functions: We systematically investigate individual reward component contributions through controlled ablation experiments. Table IV presents performance degradation when each component is removed.

TABLE IV: Ablation Results of Reward Functions

Configuration	BSR (%) ↑	HTE (deg) ↓	VTE (m/s) ↓
Complete Reward	99.90	**1.15**	**0.18**
No Survival Reward	**99.99** (+0.09)	1.27 (+0.12)	0.19 (+0.01)
No Velocity Tracking	96.71 (-3.19)	**1.15** (-0.00)	0.59 (+0.41)
No Heading Tracking	99.87 (-0.03)	6.30 (+5.15)	0.21 (+0.03)
No Action Penalty	**99.99** (+0.09)	1.72 (+0.57)	0.38 (+0.20)
No Rate Penalty	99.87 (-0.03)	2.29 (+1.14)	0.55 (+0.37)

The ablation analysis reveals that tracking rewards are crucial. Removing the velocity tracking reward leads to a significant stability performance drop (BSR -3.19%, VTE +0.41 m/s); the heading tracking reward follows a similar pattern. Interestingly, removing the survival reward and action penalty slightly improves the BSR, suggesting a potential conservative bias. Finally, the absence of the rate penalty harms control smoothness, increasing both HTE and VTE.

2) Impact Assessment of Domain Randomization: Table VI evaluates the contribution of different randomization strategies.

The analysis reveals the significant impact of different randomization strategies. The Full Randomization strategy achieves the best overall performance with the lowest tracking errors (HTE 1.15 deg, VTE 0.18 m/s). Single-component randomizations, e.g., Dynamics-Only, Initial States-Only or Terrain-Only, also result in substantial performance loss, particularly in heading and velocity control.

Interestingly, the "No Randomization" strategy yields a marginally higher BSR (99.99%) in ideal simulation conditions compared to the full strategy, indicating a performance-robustness trade-off. The extensive randomization required for sim-to-real transfer inherently introduces conservatism into the policy, slightly sub-optimizing peak theoretical performance to guarantee broad adaptability. However, as noted by the severe degradation in precision (HTE +4.01°, VTE +0.31 m/s) when deploying without full randomization, this slight conservative bias is a necessary and acceptable compromise for safe and effective deployment across varying real-world conditions.

3) Sensitivity Analysis of Reward Weights: To validate our reward configuration, we conducted a sensitivity analysis [29] by independently scaling each weight (λ_{surv}, λ_{vel}, λ_{head}, λ_{act}, λ_{rate}) from 0.1× to 5.0× of its nominal value.

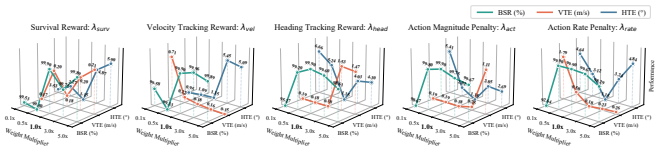


Fig. 6: Sensitivity Analysis of Reward Weights.

As illustrated in Fig. 6, the nominal configuration (1.0×) generally achieves the most balanced performance across BSR, HTE, and VTE. Deviating from these values degrades overall performance; for instance, overly prioritizing velocity tracking notably increases heading errors, while insufficient action rate penalties compromise balancing stability. Conversely, survival reward deviation leads to limited performance degradation since it primarily accelerates convergence and stabilizes training rather than dominating final performance. These results

TABLE V: Robustness Evaluation

Test Category	Condition	BSR (%) ↑	BRT (s) ↓	HTE (deg) ↓	VTE (m/s) ↓	Notes
Velocity Range	1.0-3.0 m/s	99.49	1.58	1.57	0.23	Low speed
	3.0-5.0 m/s	86.21	2.05	1.56	0.28	High speed
Sensor Degradation	No noise	**99.90**	**1.05**	**1.15**	**0.18**	Baseline performance
	Maximum noise level	96.38	2.90	3.46	0.78	20% amplitude noise
	Intermittent dropouts	90.84	1.62	1.92	0.48	10% data loss rate
Terrain Type	Flat plane	**99.90**	**1.05**	**1.15**	**0.18**	Ideal conditions
	Rough pavement	88.47	1.66	1.44	**0.18**	Standard road surface
	Gravel surface	85.87	2.15	1.69	0.21	Reduced traction
	Slope terrain	92.05	1.74	1.35	0.23	Slope adaptation
	Gentle steps	76.60	1.80	1.31	0.22	Complex terrain

TABLE VI: Component Analysis of Domain Randomization

Randomization Strategy	BSR (%) ↑	HTE (deg) ↓	VTE (m/s) ↓
Full Randomization	99.90	**1.15**	**0.18**
No Randomization	**99.99** (+0.09)	5.16 (+4.01)	0.49 (+0.31)
Dynamics-Only	99.95 (+0.05)	2.49 (+1.34)	0.43 (+0.25)
Initial States-Only	**99.99** (+0.09)	4.03 (+2.88)	0.57 (+0.39)
Command-Only	99.19 (-0.71)	1.72 (+0.57)	0.20 (+0.02)
Terrain-Only	**99.99** (+0.09)	3.84 (+2.69)	0.58 (+0.40)

substantiate that the chosen weights are empirically robust and represent an optimal operating point, effectively addressing the heuristic nature of reward shaping.

F. Implementation and Validation of Hardware System

1) Construction of Physical Platform: Our hardware validation utilizes a custom-designed bicycle platform, tailored for real-world evaluation, as shown in Fig. 7. Specifications of this hardware are detailed in Table VII.

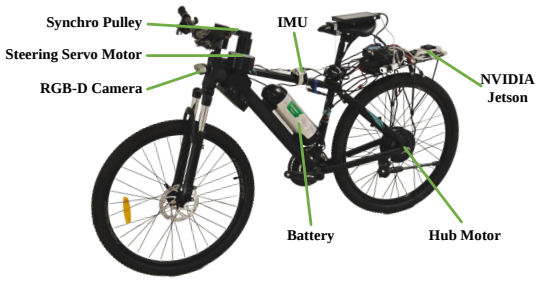


Fig. 7: Construction of hardware platform.

The mechanical design emphasizes modularity and reliability, utilizing off-the-shelf components to ensure reproducibility. Computationally, the NVIDIA Jetson board provides sufficient processing power for real-time neural network inference while maintaining power efficiency for extended operation.

2) Validation of Real-World Deployment: Real-world validation was conducted to evaluate the policy's performance and transferability. Systematic comparisons between simulation and real-world performance are shown in Table VIII.

The results confirm an effective sim-to-real transfer, achieving transfer ratios from 0.56 to 1.00 across key metrics, where a value of 1.00 indicates perfect transference. The most notable degradation, Heading Tracking Error (ratio: 0.56), is attributed to unmodeled IMU noise. Conversely, Velocity Tracking Error was nearly eliminated (ratio: 1.00), as the velocity command was directly mapped to motor speed, bypassing inference. These results validate the policy's real-world effectiveness.

Notably, real-world baseline comparisons were omitted since they failed to directly stabilize the physical platform. Under zero-shot transfer constraint, baselines were tuned solely in simulation and deployed without hardware fine-tuning, but their reliance on nominal linearized models made them highly sensitive to sim-to-real mismatches. In contrast, our domain-randomized DRL policy learned robustness to such uncertainties and successfully transferred to hardware.

To validate the policy's efficacy as a robust controller, we employed two command modalities. In the *human-in-the-loop* configuration, high-level directives ($v_{\text{cmd}}, \delta_{\text{cmd}}$) were provided by a remote operator, with the CycleRL policy autonomously managing the actuation for stability and steering. The policy was tested across diverse conditions, including asphalt roads, gravel paths, rugged grass, ramps, varying payloads and ve-

TABLE VII: Specifications of Hardware Platform

Component	Specification
Mechanical Platform	
Wheelbase	1,100 mm
Total mass	25.0 kg (including electronics)
Center of gravity height	0.65 m
Computational System	
Main processor	Jetson Orin NX 16GB
Operating system	Ubuntu 20.04
Sensing and Actuation	
IMU	WheelTec N100
Steering servo motor	Unitree GO-M8010-6
Hub motor	BaFang RM G020.500.D
Power system	48V 9.6Ah lithium battery

TABLE VIII: Comparison of Sim and Real Performance

Metric	Sim	Real	Transfer Ratio ↑
Balance Success Rate (%)	99.90	95.00	0.95
Balance Recovery Time (s)	1.05	1.29	0.81
Maximum Balance Duration (s)	>1,800	>1,800	1.00
Critical Angle Tolerance (deg)	27.79	20.12	0.72
Heading Tracking Error (deg)	1.15	2.04	0.56
Velocity Tracking Error (m/s)	0.18	0.01	1.00
System Response Latency (s)	1.06	1.10	0.96
Minimum Sustaining Speed (m/s)	1.05	1.33	0.79

locities, tire anomalies and lateral perturbations. To further demonstrate higher-level autonomy, we integrated a *vision-based lane tracking system* [30], in which RGB images from a front-mounted Intel RealSense D435i camera are used for lane detection and the deviation between the detected lane center and the vehicle's heading is fed into a tuned PID controller to yield the reference steering command. Videos of both command modalities are available at video link.

V. Conclusion

This work presented a deep reinforcement learning framework for the robust lateral control of an autonomous bicycle. By training a direct perception-to-action policy, our model-free approach successfully navigates the vehicle's complex, underactuated dynamics, offering a promising alternative to traditional model-based controllers without requiring explicit analytical dynamic modeling and system identification.

Our key contributions include a composite reward function that balances stability with tracking objectives, and a comprehensive domain randomization strategy, which proved crucial for bridging the reality gap, enabling successful transfer of the learned policy. Extensive real-world experiments validated the controller's robustness across diverse conditions, confirming its viability for autonomous cycling applications.

While our method bypasses explicit dynamic modeling, it inherently shifts the engineering effort toward simulator construction, reward shaping, and hyperparameter tuning. To address this, future work will explore automated methods to reduce these heuristic dependencies, alongside extending the framework to encompass vision-based navigation and multi-agent coordination. Ultimately, the success of this approach highlights the potential of model-free DRL in advancing intelligent mobility for sustainable urban transportation.

References

[1] T.-J. Yeh, T.-C. Lin, and A. C.-B. Chen, "Robust balancing and trajectory control of a self-driving bicycle," *IEEE Transactions on Control Systems Technology*, vol. 32, no. 6, pp. 2410–2417, 2024.

[2] S. Kuutti, R. Bowden, Y. Jin, P. Barber, and S. Fallah, "A survey of deep learning applications to autonomous vehicle control," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 2, pp. 712–733, 2020.

[3] W.-H. Huang, P. T.-T. Nguyen, D.-D. Nguyen, H.-P. Doan, M.-Y. Chuang, and C.-H. Kuo, "Stability control and path tracking of a self-balancing bicycle with a reaction wheel," *IEEE/ASME Transactions on Mechatronics*, 2025.

[4] X. Zhu, W. Xu, Z. Chen, Y. Deng, Q. Zheng, B. Liang, and Y. Liu, "Online robust self-learning terminal sliding mode control for balancing control of reaction wheel bicycle robots," *Robotica*, vol. 42, no. 10, pp. 3416–3430, 2024.

[5] W. Deng, S. Moore, J. Bush, M. Mabey, and W. Zhang, "Towards automated bicycles: Achieving self-balance using steering control," in *Dynamic Systems and Control Conference*, vol. 51906. American Society of Mechanical Engineers, 2018, p. V002T24A012.

[6] M. Baquero-Suárez, J. Cortés-Romero, J. Arcos-Legarda, and H. Coral-Enriquez, "A robust two-stage active disturbance rejection control for the stabilization of a riderless bicycle," *Multibody System Dynamics*, vol. 45, no. 1, pp. 7–35, 2019.

[7] Y. Yu and M. Zhao, "Steering control for autonomously balancing bicycle at low speed," in *2018 IEEE International Conference on Robotics and Biomimetics (ROBIO)*. IEEE, 2018, pp. 33–38.

[8] J. Pei, L. Deng, S. Song, M. Zhao, Y. Zhang, S. Wu, G. Wang, Z. Zou, Z. Wu, W. He *et al.*, "Towards artificial general intelligence with hybrid tianjic chip architecture," *Nature*, vol. 572, no. 7767, pp. 106–111, 2019.

[9] G. Bellegarda and Q. Nguyen, "Dynamic vehicle drifting with nonlinear mpc and a fused kinematic-dynamic bicycle model," *IEEE Control Systems Letters*, vol. 6, pp. 1958–1963, 2021.

[10] A. Shafiekhani, M. J. Mahjoob, and M. Akraminia, "Design and implementation of an adaptive critic-based neuro-fuzzy controller on an unmanned bicycle," *Mechatronics*, vol. 28, pp. 115–123, 2015.

[11] A. Folkers, M. Rick, and C. Büskens, "Controlling an autonomous vehicle with deep reinforcement learning," in *2019 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2019, pp. 2025–2031.

[12] V. M. et al., "Isaac gym: High performance gpu based physics simulation for robot learning," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, J. Vanschoren and S. Yeung, Eds., vol. 1, 2021.

[13] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2012, pp. 5026–5033.

[14] D. Li, D. Zhao, Q. Zhang, and Y. Chen, "Reinforcement learning and deep learning based lateral control for autonomous driving," *IEEE Computational Intelligence Magazine*, vol. 14, no. 2, pp. 83–98, 2019.

[15] C.-L. Hwang, H.-M. Wu, and C.-L. Shih, "Fuzzy sliding-mode underactuated control for autonomous dynamic balance of an electrical bicycle," *IEEE Transactions on Control Systems Technology*, vol. 17, no. 3, pp. 658–670, 2009.

[16] A. Weißmann and D. Görges, "An empirical study on ego vehicle trajectory prediction for bicycles in urban environments based on conditional imitation learning," in *2021 IEEE International Intelligent Transportation Systems Conference*. IEEE, 2021, pp. 1482–1489.

[17] H. Liu, B. Kiumarsi, Y. Kartal, A. Taha Koru, H. Modares, and F. L. Lewis, "Reinforcement learning applications in unmanned vehicle control: A comprehensive overview," *Unmanned Systems*, vol. 11, no. 01, pp. 17–26, 2023.

[18] J. Baltes, G. Christmann, and S. Saeedvand, "A deep reinforcement learning algorithm to control a two-wheeled scooter with a humanoid robot," *Engineering Applications of Artificial Intelligence*, vol. 126, p. 106941, 2023.

[19] V. Wiberg, E. Wallin, T. Nordfjell, and M. Servin, "Control of rough terrain vehicles using deep reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 7, no. 1, pp. 390–397, 2022.

[20] L. Guo, H. Lin, J. Jiang, Y. Song, and D. Gan, "Combined control algorithm based on synchronous reinforcement learning for a self-balancing bicycle robot," *ISA Transactions*, vol. 145, pp. 479–492, 2024.

[21] S. Höfer, K. Bekris, A. Handa, J. C. Gamboa, M. Mozifian, F. Golemo, C. Atkeson, D. Fox, K. Goldberg, J. Leonard *et al.*, "Sim2real in robotics and automation: Applications and challenges," *IEEE Transactions on Automation Science and Engineering*, vol. 18, no. 2, pp. 398–400, 2021.

[22] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2017, pp. 23–30.

[23] P. Zhai, X. Wei, T. Hou, X. Ji, Z. Dong, J. Yi, and L. Zhang, "Robust proximal adversarial reinforcement learning under model mismatch," *IEEE Robotics and Automation Letters*, vol. 9, no. 11, pp. 10 248–10 255, 2024.

[24] J. Truong, S. Chernova, and D. Batra, "Bi-directional domain adaptation for sim2real transfer of embodied navigation agents," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 2634–2641, 2021.

[25] J. P. Meijaard, J. M. Papadopoulos, A. Ruina, and A. L. Schwab, "Linearized dynamics equations for the balance and steer of a bicycle: a benchmark and review," *Proceedings of the Royal society A: mathematical, physical and engineering sciences*, vol. 463, no. 2084, pp. 1955–1982, 2007.

[26] N. Persson, M. C. Ekström, M. Ekström, and A. V. Papadopoulos, "Trajectory tracking and stabilisation of a riderless bicycle," in *2021 IEEE International Intelligent Transportation Systems Conference (ITSC)*. IEEE, 2021, pp. 1859–1866.

[27] S. Yasuda, T. Kumagai, and H. Yoshida, "Safe and efficient dynamic window approach for differential mobile robots with stochastic dynamics using deterministic sampling," *IEEE Robotics and Automation Letters*, vol. 8, no. 5, pp. 2614–2621, 2023.

[28] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv:1707.06347*, 2017.

[29] J. Xiong, R. Yu, and C. Liu, "Steering control and stability analysis for an autonomous bicycle: part ii experiments and a modified linear control law," *Nonlinear Dynamics*, vol. 112, no. 5, pp. 3107–3132, 2024.

[30] Z. Qin, H. Wang, and X. Li, "Ultra fast structure-aware deep lane detection," in *European conference on computer vision*. Springer, 2020, pp. 276–291.